

A Computation of the Ninth Dedekind Number using FPGA Supercomputing

LENNART VAN HIRTUM, Paderborn Center for Parallel Computing, Paderborn University, DE

PATRICK DE CAUSMAECKER, KU Leuven, Department of Computer Science, CODES research group, BE

JENS GOEMAERE, KU Leuven, Department of Computer Science, CODES research group, BE

TOBIAS KENTER, Paderborn Center for Parallel Computing, Paderborn University, DE

HEINRICH RIEBLER, Paderborn Center for Parallel Computing, Paderborn University, DE

MICHAEL LASS, Paderborn Center for Parallel Computing, Paderborn University, DE

CHRISTIAN PLESSL, Paderborn Center for Parallel Computing, Paderborn University, DE

This manuscript makes the claim of having computed the 9th Dedekind number, D(9). This was done by accelerating the core operation of the process with an efficient FPGA design that outperforms an optimized 64-core CPU reference by 95×. The FPGA execution was parallelized on the Noctua 2 supercomputer at Paderborn University. The resulting value for D(9) is

286386577668298411128469151667598498812366

. This value can be verified in two steps. We have made the data file containing the 490M results available, each of which can be verified separately on CPU, and the whole file sums to our proposed value. The paper explains the mathematical approach in the first part, before putting the focus on a deep dive into the FPGA accelerator implementation followed by a performance analysis. The FPGA implementation was done in RTL using a dual-clock architecture and shows how we achieved an impressive FMax of 450MHz on the targeted Stratix 10 GX 2800 FPGAs. The total compute time used was 47'000 FPGA Hours.

CCS Concepts: • **Hardware** → **Hardware accelerators**; *Arithmetic and datapath circuits*; *Combinational circuits*; • **Applied computing** → *Mathematics and statistics*; • **Mathematics of computing** → **Combinatoric problems**; **Combinatorial algorithms**; **Enumeration**.

Additional Key Words and Phrases: dedekind number, counting, combinatorics, accelerators, FPGA, supercomputing

Authors' addresses: Lennart Van Hirtum, lennart.vanhirtum@gmail.com, Paderborn Center for Parallel Computing, Paderborn University, Warburger Str. 100, Paderborn, DE, 33098; Patrick De Causmaecker, patrick.decausmaecker@kuleuven.be, KU Leuven, Department of Computer Science, CODES research group, KULAK, Etienne Sabbelaan 53, Kortrijk, BE, 8500; Jens Goemaere, jens.goemaere@kuleuven.be, KU Leuven, Department of Computer Science, CODES research group, KULAK, Etienne Sabbelaan 53, Kortrijk, BE, 8500; Tobias Kenter, kenter@uni-paderborn.de, Paderborn Center for Parallel Computing, Paderborn University, Warburger Str. 100, Paderborn, DE, 33098; Heinrich Riebler, heinrich.riebler@uni-paderborn.de, Paderborn Center for Parallel Computing, Paderborn University, Warburger Str. 100, Paderborn, DE, 33098; Michael Lass, michael.lass@uni-paderborn.de, Paderborn Center for Parallel Computing, Paderborn University, Warburger Str. 100, Paderborn, DE, 33098; Christian Plessl, christian.plessl@uni-paderborn.de, Paderborn Center for Parallel Computing, Paderborn University, Warburger Str. 100, Paderborn, DE, 33098.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2018 Association for Computing Machinery.

0004-5411/2018/8-ART111 \$15.00

<https://doi.org/XXXXXXX.XXXXXXX>

ACM Reference Format:

Lennart Van Hirtum, Patrick De Causmaecker, Jens Goemaere, Tobias Kenter, Heinrich Riebler, Michael Lass, and Christian Plessl. 2018. A Computation of the Ninth Dedekind Number using FPGA Supercomputing. *J. ACM* 37, 4, Article 111 (August 2018), 29 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 INTRODUCTION

Calculating Dedekind numbers (OEIS series A000372 [11]) has always followed progress in both the mathematical domain, and the computational implementation domain. Computing subsequent numbers is known as “Dedekind’s Problem”. Since the year 1991 and up until now, the highest known term in this sequence was $D(8)$, computed by Doug Wiedemann[22] on a Cray 2 supercomputer. In this paper, we’ll present our computation of the 9th instance in this sequence, and explain the steps it took, both mathematical and our FPGA implementation. This computational endeavour originally started as the master’s thesis of the main author[8], and through collaboration with the people at the Paderborn Center for Parallel Computing we were able to make it a reality in the subsequent two years.

Fundamentally, the Dedekind Sequence grows at a rate of $O(2^{2^n})$. Also, with all algorithms known to date, the fundamental complexity is bounded by $O(2^{2^{n-1}})$. In this work, we do not break this complexity barrier, but by efficiently making use of the symmetries inherent in the problem, and building an optimized FPGA accelerator for the problem, we have been able to attain the required amount of work. As has happened at the time of the computations of the Dedekind numbers 5-8 over the previous century, the present computation of Dedekind number 9 hence demonstrates the power of contemporary computing hardware, in our case FPGAs, while at the same time building on mathematical and algorithmic progress.

This paper will in the first half go through the mathematics of Monotone Boolean Functions and the core formula used in the computation, followed by an in-depth explanation of the FPGA accelerator we built to accelerate the computation. Finally we describe the issues we faced during computation, and the steps we took to mitigate them.

2 MATHEMATICAL BACKGROUND

Richard Dedekind first defined the numbers $D(n)$ in 1897 [6]. Over the previous century, Dedekind numbers have been a challenge for computational power in the evolving domain of computer science. Computing the numbers proved exceptionally hard, and so far only formulas with a double exponential time complexity are known. Until recently, the largest known Dedekind number was $D(8)$. In this paper, we report on a computation of $D(9)$. Table 1 shows the known numbers, including the result of our computation. As we explain in Section 9, some uncertainty about the correctness of the number existed at the time of the first computation and we planned a verification run. This was due to the real possibility of a bit error having occurred due to Cosmic Radiation. In the meantime however, results of an independent computation were reported [10], confirming our result. Since computational methods as well as hardware implementation differ significantly between the two computations, we can conclude that the result is correct with a probability very close to 1.

A *Boolean Function* in n variables is defined as a map from n boolean inputs to a single boolean output. Since the input space is finite, we can fully represent any Boolean Function as a lookup table over all possible input combinations. In the case of a n input Boolean Function, this becomes a 2^n entry table, where each output has two options. We shall name the set of all 2^n possible inputs B_n . The total number of Boolean Functions is therefore 2^{2^n} . Throughout the implementation, we shall use 2^n bit bitvectors to represent all Boolean Functions, be they monotone or not. The binary representation of the position of each bit indicates the input values that correspond to this

D(0)	2	Dedekind (1897)
D(1)	3	Dedekind (1897)
D(2)	6	Dedekind (1897)
D(3)	20	Dedekind (1897)
D(4)	168	Dedekind (1897)
D(5)	7581	Church (1940)
D(6)	7828354	Ward (1946)
D(7)	2414682040998	Church (1965)
D(8)	56130437228687557907788	Wiedemann (1991)
D(9)	286386577668298411128469151667598498812366	Our result (2023)

Table 1. Known Dedekind numbers, OEIS series A000372 [11, 22], including our result.

R(0)	2	
R(1)	3	
R(2)	5	
R(3)	10	
R(4)	30	
R(5)	210	
R(6)	16353	
R(7)	490013148	Tamon Stephen & Timothy Yusun (2014) [20]
R(8)	1392195548889993358	Bartłomiej Pawelski (2021) [14]
R(9)	789204635842035040527740846300252680	Bartłomiej Pawelski (2023)

Table 2. Known Equivalence Class Counts, OEIS series A003182 [12]

output. As an example, take the (Monotone) Boolean Function $a \& b$. The truth table for this MBF is $ab \Rightarrow 1, b \Rightarrow 0, a \Rightarrow 0, \emptyset \Rightarrow 0$. Translating to the bitset gives 1000.

To arrive at the set of *Monotone Boolean Functions* (henceforth referred to as MBFs), we add a constraint to regular Boolean Functions, namely that of *monotonicity*. A Boolean Function is monotone if and only if for every possible configuration of inputs, when we flip any *false* input to *true*, the output will never go from *false* to *true*. An example illustrates this well, as shown in Figure 4.

Most of the mathematics in this paper derive from Patrick De Causmaecker et al's work [2–5]. In those manuscripts however, instead of Monotone Boolean Functions they use Anti-Chains, which are more compact from a set theory point of view. These representations produce equivalent structures though, and because operations on Monotone Boolean Functions are easier to understand and more efficient to implement, we opted to use these. Knowledge of Anti-Chains is not necessary to read this paper.

The set of all possible MBFs in n variables is denoted as D_n . The sizes of these sets are named *Dedekind numbers* (Usually written as $|D_n|$ or $D(n)$ ¹) and the known values in this sequence are listed in Table 1.

Table 2 shows the known numbers $R(n)$ of equivalence classes of monotone Boolean functions under permutation of the input variables. Note that the last result dates from 2023. These Equivalence Class counts are a sequence closely linked to the Dedekind numbers themselves, closely bounded

¹Other sources sometimes also use $M(n)$

as $R(n) > \frac{D(n)}{n!}$. These numbers are used to estimate the time methods in this space usually take, as a common optimization will use the symmetries in permuting the input variables.

For the set D_n of monotone Boolean functions in n variables a natural partial order $\leq$ is defined by

$$\forall \alpha, \beta \in D_n : \alpha \geq \beta \Leftrightarrow \forall X \in B_n : \beta(X) \Rightarrow \alpha(X) \quad (1)$$

This partial ordering defines a complete lattice on D_n . We denote by $\perp$ and $\top$ the smallest, respectively the largest, element of D_n :

$$\forall X \in B_n : \perp(X) = \text{false}, \top(X) = \text{true} \quad (2)$$

$$\perp(X) = \{\}, \top(X) = \{A\} \quad (3)$$

Intervals in D_n are denoted by

$$\forall \alpha, \beta \in D_n : [\alpha, \beta] = \{\chi \in D_n : \alpha \leq \chi \leq \beta\} \quad (4)$$

For $\alpha, \beta \in D_n$, the *join* $\alpha \vee \beta$ and the *meet* $\alpha \wedge \beta$ are the monotone Boolean functions defined by

$$\forall X \in B_n : (\alpha \vee \beta)(X) = \alpha(X) \text{ or } \beta(X) \quad (5)$$

$$\forall X \in B_n : (\alpha \wedge \beta)(X) = \alpha(X) \text{ and } \beta(X) \quad (6)$$

Finally, in the formulas below, a number defined for each pair $\alpha \leq \beta \in D_n$ plays an important role. We refer to this number as the *connector number* $C_{\alpha, \beta}$ of α and β . It counts the number of connected components of *true* nodes in the resulting graph when the *true* nodes in *bottom* β are removed from α . An example of this is shown in Figure 5.

It turns out that the number of solutions of

$$\chi \vee v = \beta \quad (7)$$

$$\chi \wedge v = \alpha \quad (8)$$

for $\chi, v \in D_n$ is given by $2^{C_{\alpha, \beta}}$. This is called the *P-Coefficient* also denoted as $PCoeff[4, 5]$.

Furthermore, we define several more operators that are needed further on:

- The *dual* of an MBF where both its inputs and output are inverted. The dual of α is written as an overbar: $\bar{\alpha}$. In practice this is implemented as inverting the whole bitset, and reversing the order of all the bits (Figure 2).
- *varSwap* swaps two variables which we'll call a and b , producing a new Boolean Function that behaves as if the inputs a and b were swapped. Full permutations of all variables is built upon repeated application of this operation. In practice (assuming $a > b$) this means shifting all bits whose position containing a but not b $2^a - 2^b$ bits to the right, and those containing b but not a $2^a - 2^b$ to the left. All other bits remain the same (Figure 1).
- *monotonizeUp/down*: These functions take a Boolean Function, and for any *true* value in it, activate all nodes above, and respectively below it. *monotonizeDown* creates a Monotone Boolean Function, and *monotonizeUp* creates an inverse Monotone Boolean Function. They are implemented by successively shifting right/left and or-ing the bitset with the original by $2^1, 2^2, 2^3$ up to 2^n bits (Figure 3).

3 RELATED WORK

3.1 Mathematical related work

The work in this article is built upon three papers by Patrick De Causmaecker et al. [3–5]. Another author who has recently made big strides in this field is Bartłomiej Pawelski, who has among other

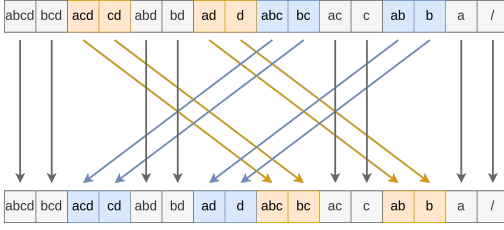


Fig. 1. varSwap example. In this example the variables b and d are swapped. This is implemented by masking out the bits of the MBF that have b but not d , and shifting them left by $2^3 - 2^1$. Likewise, we mask out bits containing d but not b and shift them right by the same amount. Other bits are kept equal.



Fig. 3. monotoneizeDown example. This takes an input Boolean Function, and finds the MBF with minimal number of *true* nodes that contains all activated input bits. It works by iteratively removing variables, and OR-ing the result with itself. Together with its upwards variant it is a core operation of the FloodFill algorithm (Section 4.1).

things extended the sequence for the Inequivalent Monotone Boolean Function counts by two terms in the last few years. [16]

Finally of course we must mention the work of Christian Jäkel, who computed the 9th Dedekind number at about the same time as us, but using different formulas and GPUs instead of FPGAs[10]. The underlying method employed in his paper is in the same family of “Jumping Formulas” as ours. While we use a 2-jump formula (Equation 9), Jäkel used a 4-jump. (Theorem 8 in [10]). Now in practice, there is not that much to gain with differing jump sizes, as the exponential decrease in $|D_n|$ due to shrinking n is almost equally balanced out by an exponential increase in the number of

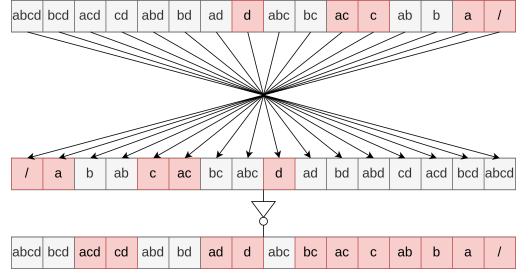


Fig. 2. dual of MBF. In this example, the red bits represent *true* values. To take the dual of an MBF, we reverse the bitset and invert it. In effect, this ‘reversing’ inverts the set of inputs for each bit.

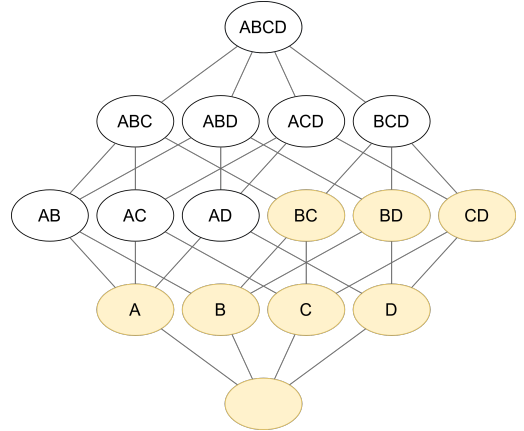


Fig. 4. An example of a 4-input Monotone Boolean Function. Each of the nodes represents the output of the Boolean Function (yellow for *true*, white for *false*). The letters in each node represent the inputs that must be *true* (and absent ones are *false*) in order to produce this output. (Originally first shown in [8])

recursive summations that must be made over this smaller set. To illustrate, for a jump of 1, we need to iterate through $D(n-1)$ MBFs once, for a jump of 2, we need to iterate through all 2-tuples of all $D(n-2)$ MBFs, 4 implies iterating through all 8-tuples of all $D(n-4)$ MBFs.

Jäkel's implementation used Nvidia A100 GPUs and took 5311 hours in GPU time, compared to 47000 FPGA hours used for our project. Despite the higher power consumption per GPU, this makes Jäkel's project more efficient in terms of energy consumed and probably also in terms hardware investments, even though the utilized FPGAs were first deployed in 2018 and thus might already be completely written off in a commercial setting. However, it is hard to attribute this difference to any specific aspect of the two projects, since the hardware involved, the algorithms employed, and the platform advantages capitalized on are so different. Jäkel was able to leverage the GPU hardware so efficiently, because he was able to reformulate this 8-way tuple summation in terms of 4,257,682,565 large matrix multiplications. This way, the GPUs were executing a task they were specifically designed for on the architecture level. Our approach on the other hand employed the reconfigurability of FPGAs to implement fixed structure boolean operations, a task which they are uniquely well suited, but not specialized for – for example our design leaves the DSP blocks, a specific strength of the Stratix 10 architecture, entirely unused. One might be tempted to attribute the difference in execution time to the much lower clock frequency of FPGAs or more generally to the perspective that GPUs in some regards resemble an ASIC specialized for matrix multiplication, in contrast to the larger versatility of the FPGA architecture. However, that would unjustly ignore the differences on multiple levels of algorithmic optimizations. All in all, each approach used their respective platform to the fullest, and achieved an impressive result that had not been reached for over thirty years.

3.2 FPGA Accelerators with Similar Characteristics

FPGA accelerators are still a relatively niche platform in the High Performance Computing (HPC) space. Their high purchase cost, steep development difficulty, and relatively low clock frequency make them a challenging target compared to traditional HPC computing platforms such as CPU and GPU computing. Nevertheless, FPGAs excel at certain specific types of tasks. Particularly high acceleration can be expected for tasks in which bit-level operations are key, or which employ an unpredictable branching structure, both of which are characteristics of this work. A domain where FPGAs are often employed due to their ability to implement bit-level operations efficiently is cryptography, for example for the calculation of MD5 hashes [7], or for searching [18] and reconstructing [17, 19] AES keys from decayed memory. Another recent example of FPGA acceleration with a focus on bit-level operations is mutation tree reconstruction [13] in bioinformatics.

4 WORKLOAD CHARACTERIZATION AND APPROACH

We start from the original PCoeff Formula as taken from [4]:

$$D(n+2) = \sum_{\substack{\alpha, \beta \in D_n \\ \alpha \geq \beta}} |[\alpha, \top]| 2^{C_{\beta, \alpha}} |[\perp, \beta]| \quad (9)$$

In the master thesis of the first author of the current paper, Lennart Van Hirtum [8], the author reworked this formula to a form making use of equivalence classes to reduce the total number of

terms.

$$D(n+2) = \sum_{\alpha \in R_n} |[\alpha, \top]| E_\alpha \sum_{\substack{\beta \in R_n \\ \exists \delta \in \text{Permut}_\beta: \alpha \geq \delta}} |[\perp, \beta]| \frac{E_\beta}{n!} \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \geq \gamma}} 2^{C_{\gamma, \alpha}} \quad (10)$$

The Permut_β term is the collection of all $n!$ equivalents of β under permutation of the input variables. E_β is the number of distinct equivalents, and hence, Permut_β contains duplicates iff $E_\beta < n!$. The number of duplicates of any given permutation is $\frac{n!}{E_\beta}$ and are divided out by the $\frac{E_\beta}{n!}$ factor. Technically the condition isn't required on the sum over all β , since any β s that don't will have zero valid γ terms. It is included here however, because the implementation efficiently enumerates these β s by iterating through the Equivalence Class graph shown in Figure 12.

$$\#PCoeff(n+2) = \sum_{\substack{\alpha, \beta \in D_n \\ \alpha \geq \beta}} 1 = \sum_{\alpha \in D_n} |[\perp, \alpha]| \quad (11)$$

For $D(9)$, this means iterating through D_7 . That would require iterating over an estimated 4.59×10^{16} α, β pairs [8, Section 9.1]. A lower bound for the total number of P-Coefficients ($2^{C_{\alpha, \gamma}}$) that need to be computed can be calculated exactly by counting the number of terms in Equation 9. This is a lot of work still though, so we reworked the formula to Equation 11 giving us $11483553838459169213 = 1.148 \times 10^{19}$. This is only a lower bound, as iterating over *all permutations* as Equation 10 does, introduces some redundancy to allow for a simpler implementation. However it is a pretty tight bound, as the number of Equivalence Classes that doesn't have the full 5040 distinct permutations is only 4.3% (See table 2).

We were able to improve on this further using the process of 'deduplication', where we can halve the total amount of work again, by noticing that pairs of α, β give identical results to their dual pair $\bar{\beta}, \bar{\alpha}^2$. As per Equation 12. This allowed us to halve the total amount of work to 5.574×10^{18} P-Coefficients³.

From now on when we refer to a 'top' or a 'bottom', we will be referring to the α , respectively β in equation 10. When referring to 'bottom permutations' we are referring to γ .

$$\begin{aligned} |[\alpha, \top]| 2^{C_{\beta, \alpha}} |[\perp, \beta]| &= |[\perp, \bar{\alpha}]| 2^{C_{\beta, \alpha}} |[\bar{\beta}, \top]| \\ &= |[\perp, \bar{\alpha}]| 2^{C_{\bar{\alpha}, \bar{\beta}}} |[\bar{\beta}, \top]| \end{aligned} \quad (12)$$

4.1 Computing P-Coefficients on FPGA

Computing P-Coefficients requires solving the problem of counting the number of disjoint connected components within the activated nodes of an $(n-2)$ -dimensional hypercube. An example of such a graph with its disjoint connected components colored is shown in Figure 5 for five variables. For calculating $D(9)$, corresponding graphs for seven variables are used, which contain 128 vertices and 448 edges. Even though the graph structure itself is fixed and only the set of active vertices changes for every P-Coefficient, a typical CPU implementation will contain many branches and it's hard to make use of vectorization based on the Single Instruction, Multiple Data (SIMD) paradigm. This observation motivated the use of FPGAs in this work, since counting connected components

²This follows immediately from the definition of the PCoef in equation 7

³We made sure not to deduplicate pairs that were their own dual, ie when $\beta = \bar{\alpha}$

Class Size (E_α)	Classes of this size ($\#\alpha$)
5040	468822749
2520	20005503
1680	3128
1260	1024050
840	75024
720	4
630	47242
504	237
420	26739
360	18
315	2742
252	114
210	3255
140	702
120	4
105	1206
84	9
70	90
42	99
35	117
30	5
21	75
7	27
1	9

Table 3. MBF lookup table by Equivalence Class Size for MBFs of 7 Variables. The second column sums to $R(7)$. The sum of the products of the columns gives $D(7)$

involves only plain Boolean operations along the graph edges, which can be mapped extremely well to FPGA technology. Because the overall structure of the graph is fixed, the wires and gates resemble the edge and node structure of the graph. A simple schematic implementation of a *Count Connected Core* is shown in Figure 6. An implementation of the Count Connected Core with first estimates about resource consumption and possible parallelism was first proposed in [8]. In the thesis, two preprocessing steps - Singleton Elimination and Leaf Elimination[8, Section 9.3] - are derived that bring the average number of iterations (and thus clock cycles) down to 4.061. These are described further in Section 5.3. In this manuscript, we expand upon this and describe a complete FPGA accelerator in Section 5, with further details about counting connected components on FPGA and with optimized CPU code in Section 5.3 and further performance analysis in Section 7.

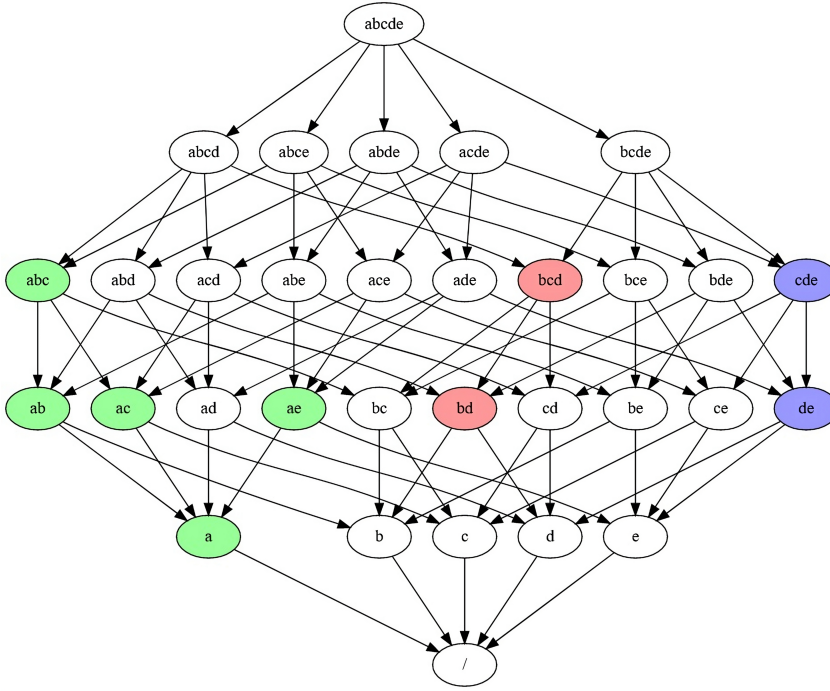


Fig. 5. Connected components of an example graph. This graph could crop up when for instance the given top MBF α was all colored nodes, plus all nodes below them required to form a valid MBF, and the bottom MBF γ would have been just the white nodes added to α . In this case there are 3 connected components.

Count Connected Core

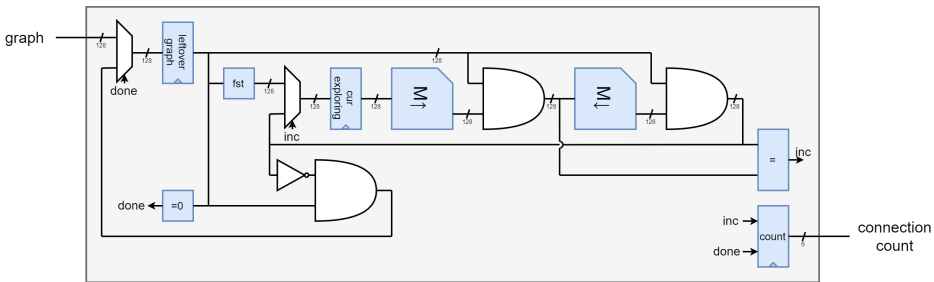


Fig. 6. Schematic representation of the core component in the FPGA Accelerator: The Count Connected Core. This is the hardware equivalent of the software code from Listing 4. The main components of the algorithm can be seen here: The monotoneUp/down, new seed finding, and subtracting found components from the graph. It should be noted that only the second check for breaking the loop is present. This is because having the extra check in software constitutes an early break, but this optimization doesn't apply to hardware. We can safely remove it, because detecting that the exploration step is finished in the first check, necessitates detecting it in any subsequent checks.

5 ACCELERATOR IMPLEMENTATION

Since the Count Connected Core is the element which enables the large speedup of our FPGA computation, our goal is to fit as many of these on a single FPGA as possible, and run them at an as high as possible clock frequency. At this rate however, it also becomes an important concern to create a design that can keep up with Count Connected Core's data consumption and result production rate. Furthermore the PCIe communication bandwidth has to be taken into account to deliver work items fast enough.

To parallelize a total of 300 Count Connected Cores per FPGA, while staying aware of these constraints, we divide them into a hierarchy. Groups of 30 count connected cores form work units called *Full Permutation Pipelines* (FPP), and further parallelization and load balancing happens between these 30 core groups.

In the subsequent parts of this section, we will go down the hierarchy. We will start at the top-level OpenCL glue code level in Section 5.1 which deals with the input and output buffering, DDR access, and reordering the bottoms to balance the throughput on each component. Section 5.2 deals with the top-level Verilog Module. It handles the load-balancing over the 300 Count Connected Cores. At the top level it load-balances the bottoms stream over the 10 Full Permutation Pipelines. Each of these 10 FPPs consists of 30 Count Connected Cores each responsible for 1/30th of the 5040 permutations of the bottom. Finally, in Section 5.3 we delve into the actual implementation of these Count Connected Cores, how the iteration structure works, and how they are optimized for speed. Results of actually synthesizing the design are presented in Section 5.4.

A pseudocode overview of the algorithm is given in Listing 1.

```
# Iterate over all tops. Parallelized across compute jobs
for  $\alpha \in R_n$ :
    # Contains only  $\beta$  which have valid permutation (Section 5.5)
    # This reduces the amount of PCIe data transmission.
    # Also deduplicate over duals
    Construct buffer  $B_\alpha$  of all  $\beta \in R_n : \beta \leq_T \bar{\alpha} \wedge \exists \gamma \in \text{Permut}_\beta : \gamma \leq \alpha$ .
    Transmit  $B_\alpha$  to FPGA
    # Parallelize over 10 Full Permutation Pipeline (Section 5.2)
    for  $\beta \in B_\alpha$ :
        # Parallelize over 30 Pipelines per FPP (Figure 8)
        # For each permutation of bottom  $\beta$ 
        for  $\gamma \in \text{Permut}_\beta : \gamma \leq \alpha$ :
            Compute  $C_{\alpha,\gamma}$  # (Section 5.3)
             $P_{\alpha,\gamma} = 2^{C_{\alpha,\gamma}}$ 
            sum  $P_{\alpha,\gamma}$  to create  $\Sigma P_{\alpha,\beta}$ 
            Gather all resulting  $\Sigma P_{\alpha,\beta}$  in result buffer  $R_\alpha$ 
        Transmit  $R_\alpha$  back to host
    for  $\beta \in B_\alpha$ :
         $\text{dup} = \beta == \bar{\alpha} ? 1 : 2$ 
        sum  $||[\perp, \beta]|| \frac{E_\beta}{n!} \Sigma P_{\alpha,\beta} \text{dup}$  over all  $\beta$  to create  $\Sigma_\alpha$  # (Equation 10)
        sum  $||[\alpha, \top]|| E_\alpha \Sigma_\alpha$  over all  $\alpha$  to create  $D(n)$ 
```

Listing 1. High level overview. References are given to sections that delve into how each part is implemented.

5.1 OpenCL interface

The OpenCL code is actually a rather simple wrapper around the Verilog code, as shown in Listing 2:

```

kernel void dedekindAccelerator(
    global const ulong2 * restrict mbfLUTA,
    global const ulong2 * restrict mbfLUTB,
    global const uint2 * restrict jobsIn,
    global ulong2 * restrict processedPCoeffsOut,
    uint workGroupSize
) {
    // 2 mbfs per cycle to meet throughput
    for (uint i = 0; i < workGroupSize / 2; i++) {
        uint shuffledI = shuffleClusters(i, workGroupSize / 2);
        uint2 curJob = jobsIn[shuffledI];
        // First bit marks the start of a new bot series
        bool startNewTop = (curJob.x & 0x80000000u) != 0x00000000u;
        // Cut off the first bit, so we don't have incorrect addresses
        ulong2 mbfA = mbfLUTA[curJob.x & 0x7FFFFFFFu];
        ulong2 mbfB = mbfLUTB[curJob.y & 0x7FFFFFFFu];

        ulong2 result = pcoeffProcessor(mbfA, mbfB, startNewTop);
        processedPCoeffsOut[shuffledI] = result;
    }
}

```

Listing 2. Main kernel of the FPGA accelerator. This is simply a wrapper around the core logic implemented in Verilog. This wrapper takes care of feeding data into the verilog module, and storing the results. It reads in data at two bottoms per cycle, and it shuffles the order of the bottoms to be processed, to more evenly intermix easy and difficult bottoms to diffuse the load on the different parts of the Verilog implementation.

We iterate over the given 32-bit MBF indices, look them up in our MBF Lookup Table (MBF LUT), and run the PCoeff processor on them. Because the interface to the Verilog code can't be called in a different way for the top as for the bottom, we pass the top to the module first, with the *startNewTop* bit '1'. To further simplify the hardware we store this bit as part of the index. We just use a free bit from the index which is '1' for the first element, and '0' for all others. We took care that *shuffleClusters* doesn't reorder this first element, so that it always gets passed to *pcoeffProcessor* first.

Each result is then stored in the output buffer. The reason we use a lookup table instead of sending the MBFs over directly is to limit PCIe bandwidth use. If we only send indices, we only need to send 32-bit indices, instead of the full 128-bit MBFs. We use two copies of the lookup table to parallelize our accesses over 2 of the 4 memory banks. The results are returned in 64-bit buffers, of which the first 48 bits contain the sums for each β , Equation 13. The next 13 bits are used for the number of valid permutations Equation 14 (this turned out to be very useful in Section 6). Finally one more bit is used to flag detected bit errors in the FPGA BRAMs.

$$P_{sum,\alpha,\beta} = \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \geq \gamma}} 2^{C_{\gamma,\alpha}} \quad (13)$$

$$P_{count,\alpha,\beta} = \sum_{\substack{\gamma \in \text{Permut}_\beta \\ \alpha \geq \gamma}} 1 \quad (14)$$

We run the function on two MBFs at a time, to double our bandwidth, as OpenCL will compile this to a module processing one iteration per clock cycle.

The *shuffleClusters* call shuffles the order in which we iterate through the MBF list to keep the computational difficulty reasonably even. That is because the elements at the front of the buffer tend to be high MBFs, of which only a small fraction of the 5040 permutations is valid, whereas the end of the buffer has the bottom most MBFs, and pretty much all of these have all 5040 valid permutations. A consequence is that the computational difficulty generally rises from front to back. If implemented straightforwardly, at first, the computation module will be underutilized and the bottleneck is in permutation checking whereas towards the end of the list the computation module will be overutilized. This is alleviated by alternating easy and difficult bottoms leading to a relatively even difficulty across the whole run. The implementation we used for it is shown in Listing 3.

```

uint bitReverse4(uint v) {
    return ((v & 0x1) << 3) | ((v & 0x2) << 1)
           | ((v & 0x4) >> 1) | ((v & 0x8) >> 3);
}

uint shuffleIndex(uint idx, uint workGroupSize) {
    uint chunkOffset = workGroupSize >> 4;
    uint mod = idx & 0xF;
    uint div = idx >> 4;
    return chunkOffset * bitReverse4(mod) + div;
}

uint shuffleClusters(uint idx, uint workGroupSize) {
    uint block = idx >> 4;
    uint idxInBlock = idx & 0xF;
    uint numberOfBlocks = workGroupSize >> 4;
    return (shuffleIndex(block, numberOfBlocks) << 4) | idxInBlock;
}

```

Listing 3. Implementation of *shuffleClusters* which is quite primitive as to not consume too much hardware. It divides the given buffer into 16 equal chunks, and intersperses blocks from them. First it reads 16 elements from chunk 0, then 16 from chunk 8, then from chunk 4, then 12, etc. The reason it reads 16 elements per block is to ensure the DDR streaming reads are still used efficiently.

5.2 PCoeff Computation Module

The outermost module divides the two incoming MBF streams across 10 “Full Permutation Pipeline” module copies we were able to fit on the FPGA. Each module takes at most one input every 7 clock

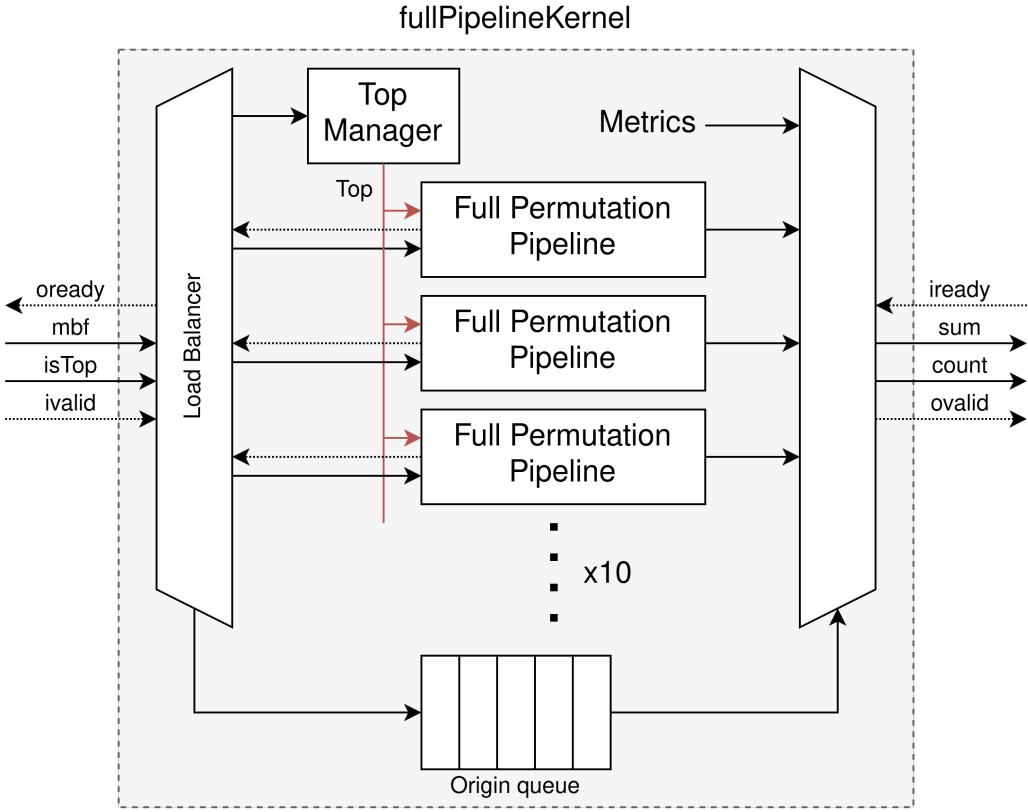


Fig. 7. Full Verilog Kernel, the first cycle a single top comes in and is stored in the Top Manager. Subsequent bottoms are fed in at a rate of two per cycle, and load-balanced across the 10 pipelines.

cycles. The outputs from the computation modules are then ordered again and returned as two output streams. This is shown in Figure 7.

Each of these modules then does the actual computational work. The actual work is to try all $5040 = 7!$ permutations of each bottom, and for the ones that are dominated by the top MBF, compute and sum their P-Coefficients. Iterating these permutations is done using 'varswap' operations. As shown in [8] a swap operation in hardware just swaps 2×32 wires out of the 128 wires. In fact, any static permutation can be done at zero cost in hardware, as it simply reorders the wires.

The strategy for iterating all permutations is recursively swapping every possible pair of variables. So variable 0 is swapped with variable 0-6, For each var 0, var 1 is swapped with variables 1-6, var 2 is swapped with variables 2-6, etc. Doing this for all variables gives us the 5040 permutations we need.

These permutation factors are split across the length of the pipeline as follows: The first 7 var swaps are iterated at the front. Permutations 6 and 5 are then implemented in wires and split across $6 * 5 = 30$ computation modules. Each of these then iterates through the remaining $4 * 3 * 2 * 1 = 24$ permutations, which of these are valid permutations, and sends only those to the Count Connected Core for processing. This is graphically shown in Figure 8. Do note there are two filtering stages, if all $4 * 3 * 2 * 1 = 24$ permutations bundle are all invalid, they are not stored in the FIFO. Then during iteration over these permutations in each module, only the valid permutations are iterated

FullPermutationPipeline 30

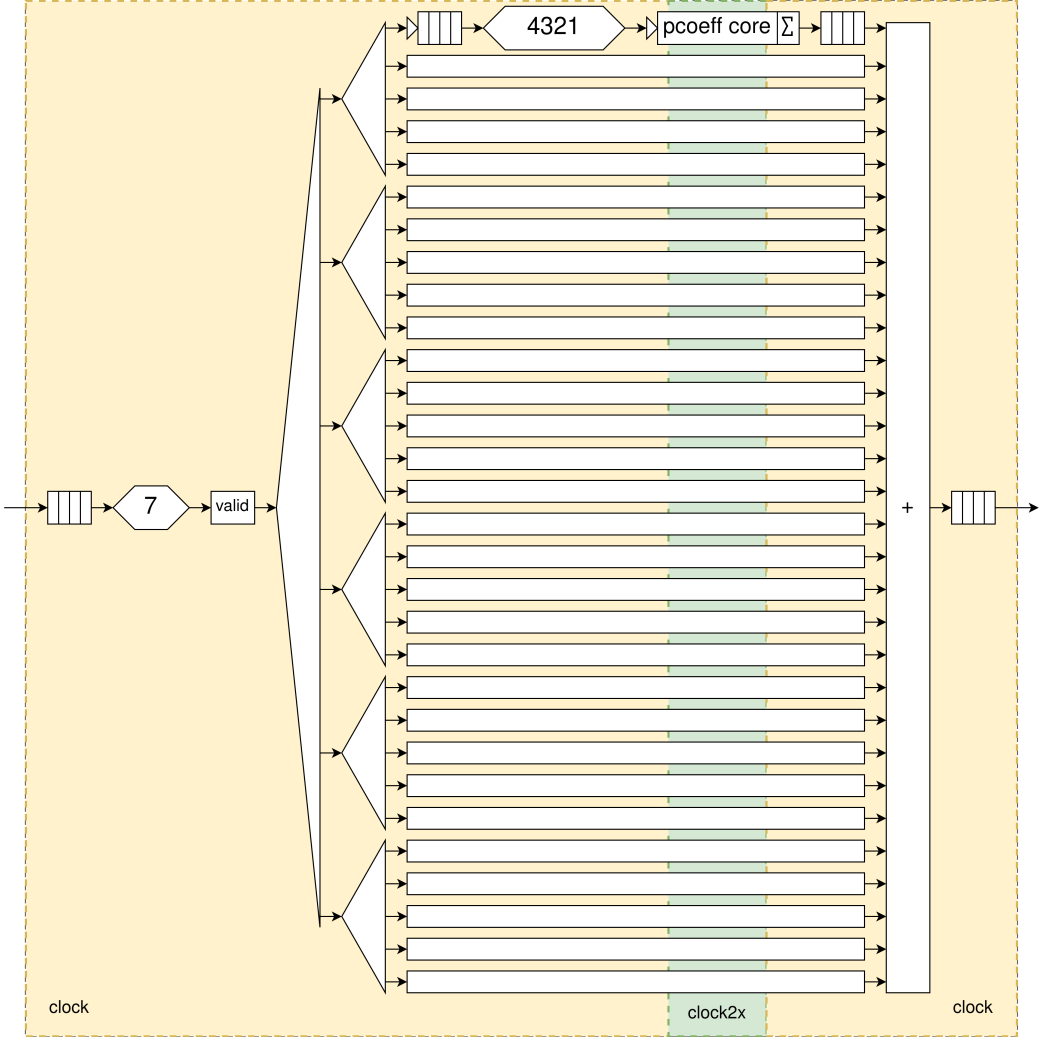


Fig. 8. Division of the permutations across 30 PCoefficient cores. Bottoms come in on the FIFO on the left, over 7 cycles the first assignments of the first variable are iterated. Then validity of all $6 * 5 * 4 * 3 * 2 * 1 = 720$ remaining permutations is computed in a single block. Each of the compute cores is responsible for $1/30$ th of these permutations. The permutations 6 and 5 are wired into the connections to the 30 computation cores and thus are free. Finally each core iterates through the valid ones of the remaining 24 permutations and feeds them into the PCoefficient Core.

through, this saves the clock cycles that would otherwise be wasted iterating through invalid ones. Checking for validity of various permutations can actually reuse a lot of hardware, which is why the validity for all $6 * 5 * 4 * 3 * 2 * 1 = 720$ permutations are checked in one large module at the front, and these validity wires are then passed to the filters of the computation pipelines.

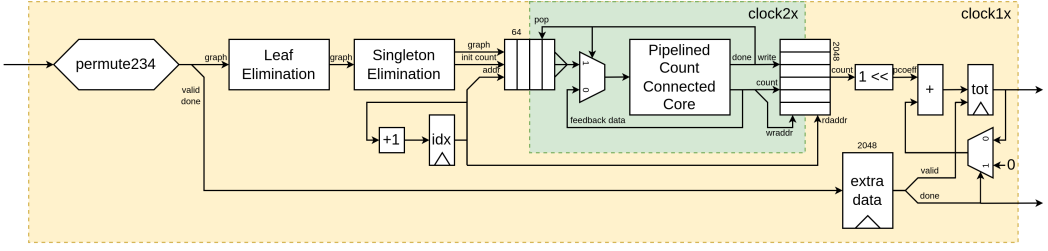


Fig. 9. PCoeff Core that wraps the Pipelined Count Connected Core shown before. It generates the 24 permutations it's responsible for, then it preprocesses each graph. It contains the feedback loop mechanism that keeps the Count Connected Core fed with new data, and the logic to put the variable latency results back in the original order. Finally it sums the resulting P-Coefficients and produces the final sum.

The results from each Count Connected Core are summed separately, and finally all 30 pipeline results are added together to produce the total P-Coefficient sum for this top-bottom combination.

5.3 Count Connected Core

Firstly, the final 4 variables are permuted, outputting only the valid permutations. (Those where the permuted $P(\beta)$ is a proper subset of α). In that case, the graph for which the number of connected components is to be computed is given by $\alpha \& P(\beta)$.

Finally we arrive at the part of the system that does actual computation, the count-connected-core. It implements a version of the Depth First Search algorithm for finding the connected components [21] which we will call the 'FloodFill' algorithm as described in [8]. It is shown in Listing 4.

The basic principle is that, so long as the remaining graph isn't empty (see Figure 5), grab an arbitrary new seed, and from there, expand the component frontier into its connected nodes until there are no new nodes to find. Due to the structure of Boolean Functions, this exploration can actually be split up efficiently into a MonotonizeUp and MonotonizeDown step. Once we can explore no further, the explored region is removed from the remaining graph, the connection count is incremented, and a new seed is chosen. This is repeated until the graph is entirely empty. The fact that the graph only consists of 128 nodes that may be activated or not, allows us to represent the whole graph as just 128 wires. The operations can operate on all nodes simultaneously, which allows us to explore many nodes in a single clock cycle.

For the CPU reference, we took great care to produce as optimal code as possible for the 7 variable case (required for computing $D(9)$). We store the vector in a 128-bit SIMD register, this way we can work on the whole vector at once. An excerpt of the monotonizeUp routine is given in Listing 5. It is basically a sequence of masked shifts and ORs. First we mask out every second bit, shift over and or with the original, then mask out 2 out of every 4, 4 out of 8, etc.

This masking and shifting is one area where the FPGA saves resources over CPU, because on the CPU we are spending valuable instructions and registers masking out bits and shifting them, whereas on FPGA this masking and shifting is done implicitly in the connecting wires. Furthermore, the FPGA can save logic blocks by merging adjacent ORs into single 4-input OR gates.

Perhaps further instruction level parallelization could be possible with 256 or 512 bit AVX registers by monotonizing multiple vectors at once, but that would be difficult to do due to the number of iterations not being constant and usually small (1-5). We didn't expect much gain from following this path.

In [8] the author described two preprocessing steps that reduce the number of iterations of the FloodFill algorithm. The first is 'Leaf Elimination'. This algorithm removes any nodes which are

connected to only a single other node. Of course that might accidentally remove a whole component if it only consists of 2 nodes. To solve that, the leafElimination comes in two kinds: Eliminating only leaves that have a single node below them, and above them respectively. It turns out, due to the fact that in the FPGA implementation the termination condition is only checked after `monotonizeDown` (See Figure 6), only `leafEliminationDown` has a significant effect on the iteration count.

The second preprocessing step is `SingletonElimination`. This step counts and removes any nodes that have no neighbors. Since on average 2.670 components are singletons, this is a significant improvement. As shown in [8, Table 9.3], these operations together significantly reduce the number of FloodFill iterations, from 6.967 down to 4.061.

The preprocessing steps both have been implemented on CPU and on FPGA. Next we consider their further integration into the FPGA design. Because we'll be pipelining these modules significantly, multiple work items will be in flight at any time, and due to the variable iteration time, they won't all finish at the same time. For this reason we include a reordering buffer at the output of the Count Connected Core. This means that every graph that goes into the core gets an ID, which is then used to index into the output memory block to store the resulting count. This memory block is then continuously read in ascending order, which puts the results back into the correct order.

```

785 // Removes any node that has no connections above it
786 // and has exactly one connection below it
787 void leafEliminationDown(BF* graph) {...}
788 // Returns the number of singletons found and removes them
789 int singletonElimination(BF* graph) {...}
790
791
792 int countConnected(BF graph) {
793     leafEliminationDown(&graph);
794     const int singletonsFound = singletonElimination(&graph);
795
796     int totalConnectedComponents = singletonsFound;
797
798     while(!graph.isEmpty()) {
799         // get seed element from an arbitrary group
800         // guaranteed to have no down-connections by picking first
801         BF expandedDown = graph.getFirst();
802         totalConnectedComponents++;
803         while(true) {
804             // add all elements connected above the current group
805             BF expandedUp = expandedDown.monotonizeUp() & graph;
806             // if no change, then we've explored the entire group
807             if(expandedUp == expandedDown) break;
808             // add all elements connected below the current group
809             expandedDown = expandedUp.monotonizeDown() & graph;
810             // if no change, then we've explored the entire group
811             if(expandedUp == expandedDown) break;
812         };
813         // remove the group we found and start looking for the next one
814         graph = graph & ~expandedDown;
815     }
816     return totalConnectedComponents;
817 }
818
819

```

Listing 4. The countConnected function as found in our github repository. ⁴ Note the highly unpredictable loop branches in this code.

⁴<https://github.com/VonTum/Dedekind/blob/main/dedelib/connectGraph.h#L278-L311>

```

834 // bs is a __m128i with the current Boolean Function
835 constexpr __m128i mask1 = _mm_set1_epi8(0b10101010);
836 constexpr __m128i mask2 = _mm_set1_epi8(0b11001100);
837 constexpr __m128i mask4 = _mm_set1_epi8(0b11110000);
838 __m128i v0Added = _mm_slli_epi64(_mm_andnot_si128(mask1, bs), 1);
839 bs = _mm_or_si128(bs, v0Added);
840 __m128i v1Added = _mm_slli_epi64(_mm_andnot_si128(mask2, bs), 2);
841 bs = _mm_or_si128(bs, v1Added);
842 __m128i v2Added = _mm_slli_epi64(_mm_andnot_si128(mask4, bs), 4);
843 bs = _mm_or_si128(bs, v2Added);
844 __m128i v3Added = _mm_slli_epi16(bs, 8);
845 bs = _mm_or_si128(bs, v3Added);
846 __m128i v4Added = _mm_slli_epi32(bs, 16);
847 bs = _mm_or_si128(bs, v4Added);
848 __m128i v5Added = _mm_slli_epi64(bs, 32);
849 bs = _mm_or_si128(bs, v5Added);
850 __m128i v6Added = _mm_slli_si128(bs, 8);
851 bs = _mm_or_si128(bs, v6Added);
852
853

```

Listing 5. Exerpt from the `monotonizeUp` function. Accelerated using SIMD instructions, where the full Boolean Function is stored in a single 128-bit AVX register. Bits are shifted by 1, 2, 4, ..., 64 positions and or-ed together. Note that masks must be used in the first three shifts, whereas the four other shifts can be optimized by exploiting their data widths. (See the `monotonizeDown` illustration in Figure 3)

You'll notice two shaded regions in Figure 9. In order to improve the processing speed of the most performance critical part of the design, we doubled the clock speed in the green region to 450MHz. This of course further extended the pipelining demands of the core pipeline, but overall was worth the required resources for the higher compute speed. The boundaries of this clock domain are the input FIFO, and the reordering memory module, which handle the clock domain crossing at their inputs and outputs natively.

A major design consideration for the reorder buffer is to make the output memory large enough, such that by the time a valid index is reached, its computation must have finished. In our case, this was proven by computing the maximum number of cycles that a new element could take from the moment it is assigned an ID and placed in the input queue, to the moment its result is written to the reorder buffer. The extreme case for this is that every MBF that needs to be processed is the worst case MBF as shown in Figure 10. It will take 20 iterations to compute it. The total pipeline depth is 20 stages. Let's say for the sake of argument that the whole pipeline and the input FIFO are filled with fresh worst-case MBFs. Our target MBF is now fed into the FIFO at the 63rd spot. (Input FIFO is 64 elements). Then it takes $20 \times 20 = 400$ cycles to resolve all MBFs in the pipeline. At this point 20 new MBFs from the queue are fed in. We need to process 5 such batches to get to and process our target MBF. This would take $20 \times 20 \times 5 = 2000$ clock_{2x} cycles. This would be cutting it close with a reorder buffer of size 2048, were it not for the fact that the indices are generated by the regular clock, which runs half as fast. At 1000 regular clock cycles max, our 2048 element reorder buffer is more than comfortable, and it happened to be a convenient size to use as this is the default depth of Stratix M20K blocks.

Figure 11 shows the time-forward combinatorial logic of the Count Connected Core. You can clearly see the three main components of the CountConnected algorithm there: Exploring new

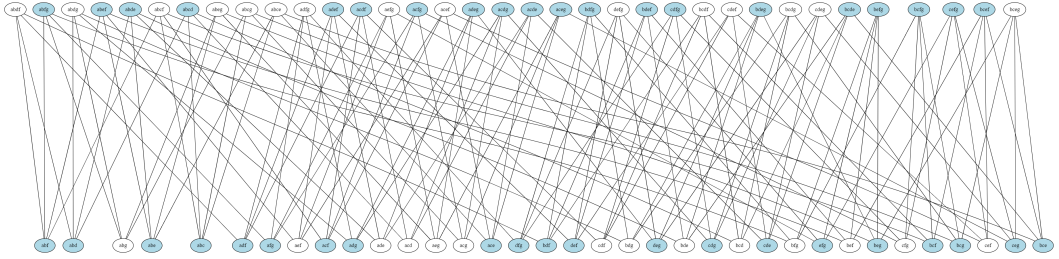


Fig. 10. Middle two layers of Worst-Case for Count Connected Algorithm. Found by computer search. Its structure is a single component where each node has two neighbors except the ends. Resolving this Component takes the algorithm 20 iterations if the algorithm starts at one of the ends.

Pipelined Count Connected Core

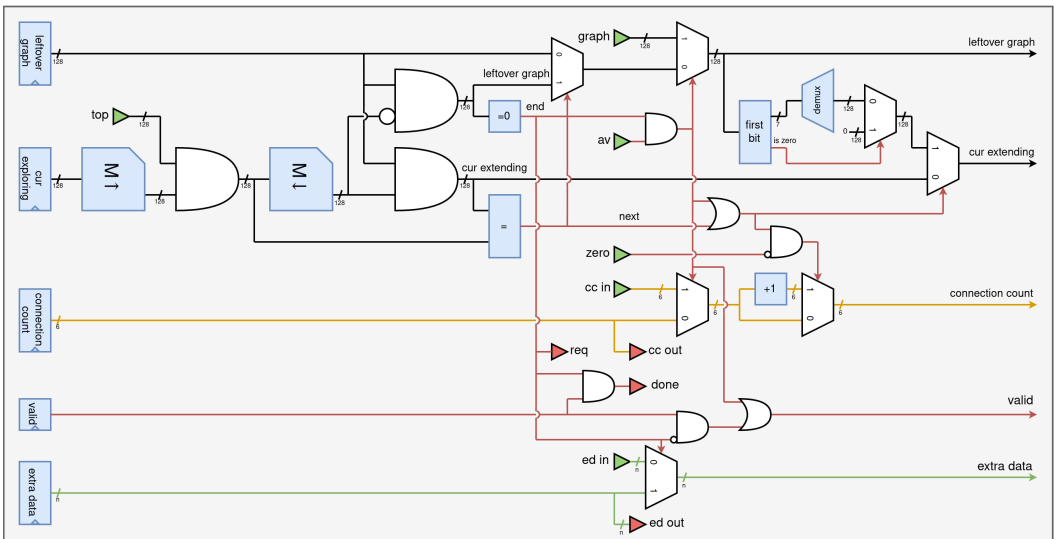


Fig. 11. This figure shows a reworked version of the Count Connected Core from Figure 6, as it is implemented in the code. The signals coming out on the right are fed back into the corresponding registers on the left. Inputs come into the system on the green arrows, and outputs are produced on the red arrows. The *top* input is special, in that it remains effectively constant for a batch of bottoms. This means we don't have to spend resources to *top* synchronized while pipelining the module. It is split into a combinatorial part, and the registers that store the state on the left edge. Describing the component like this allows us to easily pipeline it.

nodes is done with MonotonizeUp and MonotonizeDown on the left, selecting new seed nodes is done on the top right, and the iteration management hardware is in the middle. The advantage of representing it like this is that pipelining this design becomes trivial. Just draw vertical lines through the diagram, and place registers wherever one crosses the data lines.

To reach the 450Mhz frequency we need, we pipelined this module in this way quite extensively. We settled on 20 pipeline stages in the end. 10 stages for the exploration pipeline, 5 stages for the new-seed pipeline, and 5 stages of input FIFO read latency. To save on resources, as some of the

signals started using many registers for their pipeline, (~1500 registers for leftover_graph and another 500 for connection_count and output_index together), we replaced those with Block RAM shift registers.

5.4 Synthesis

The hardware accelerator was implemented on Bittware 520N cards, containing an Intel Stratix 10 GX 2800 FPGA, and 16 GB of DDR4 external RAM. It was synthesized through combining aocl's build with extra Logic Lock regions to improve the generated design's clock speed. Synthesis output is summarized in Table 4.

Name	Value	Utilization
ALUTs	1,067,787	
Registers	1,492,343	
Logic utilization	772,483 / 933,120	83 %
I/O pins	561 / 912	62 %
DSP blocks	2 / 5,760	< 1 %
Memory bits	52,971,640 / 240,046,080	22 %
RAM blocks	7,489 / 11,721	64 %
Actual clock freq	225.0	
Kernel fmax	225.425	
1x clock fmax	228.67	
2x clock fmax	450.85	
Highest non-global fanout	2332	

Table 4. Output of the synthesis process using aocl. Reported values include resources used by the Board Support Package and are reported relative to the total resources of the FPGA. Within the kernel region, more than 90% logic density is achieved.

We also performed power measurements on the synthesized FPGA kernel with the help of the onboard INA219 power monitors than can be queried using the bittware_power command in the PC2 infrastructure. The reported power consumption of the FPGA card fluctuates between 96W and 111W. The average power use is 102W. In contrast, a single AMD Epyc 7763 CPU as installed in Noctua 2 [1] already consumes 250W (not including DRAM power) when using all 64 cores for the same workload – at much lower performance, as discussed in Section 7.1.

5.5 Single Node System Integration

Finally, putting it all together, we now have a hardware accelerator module that can process on average 33 billion P-Coefficients per second, equivalent to about 275 million α, β pairs per second. We now need the capacity to feed it data rapidly enough.

We have two 520N FPGA cards per FPGA node on Noctua 2, so we need the capacity to generate 550 million β elements per second for processing. To save on bandwidth and FPGA processing time, we want to only send over α, β pairs for which we know they will have at least one valid $P(\beta) \leq \alpha$ permutation. To find these, we use a data structure called the “downlink structure” as explained in the thesis [8], Chapter 7. It divides all MBF Equivalence Classes into layers determined by their size, and then links elements from these layers together when they can be turned into each other by adding/removing a single node and re-canonizing. An example for this is shown in Figure 12. To use it for iterating over all MBFs which are lower than some given MBF α for example, one can

simply start with the canonization of α , and then iterate over all Equivalence Classes below it in a layer-by-layer fashion. Since we already have a MBF lookup table on the FPGA DDR itself, we actually only need to send the indices themselves. That saves on CPU bandwidth as well.

Finally, to get enough bandwidth, we need enough processing capacity to generate these buffers, and process the results. Each node of Noctua 2 [1] is equipped with two AMD Epyc (codename Milan) CPUs with together 128 Zen3 cores, Epyc 7763 CPUs in the compute nodes and slightly lower clocked Epyc 7713 CPUs in the FPGA nodes. Figure 13 shows the structure of tasks and data structures running on these CPUs. We chose 16 threads for the Input Producers, 1 thread to operate both FPGAs, 8 threads for processing the results, and the remaining cores were used for performing CPU validation of the FPGA results. These threads were divided equally over the 16 core complexes and 8 NUMA nodes to optimally access the system's memory.

The memory layout is carefully balanced to fit into the available 512GB of RAM available in each node. The InputProducer threads share 2 copies of the DownLink structure (each on one socket's memory) of 45 GB each, though each InputProducer also has a private 2GB for operating its Swapper for iterating through the DownLink structure. InputProducers always produce eight buffers at once, to share the memory overhead of reading the Downlink structure.

The processor threads upload the 8GB MBF Lookup Table to the FPGAs, and the validation threads store this same buffer, also in one copy for each socket. The result aggregators have another lookup table for the Interval and class size infos, also replicated per socket. But then each result aggregator also has a private 8GB Validation Buffer, the results of which are summed in the end.

All remaining memory is used for storing Input Index and Result buffers, of 1GB and 2GB each respectively. It is important to have enough buffers circulating to ensure modules don't waste time waiting for new empty buffers.

6 CHALLENGES

There were a few major challenges we encountered in developing for this FPGA platform that we deem interesting enough to share. The subtleties exposed and the solutions we had to develop are directly linked to the major challenge in this computational project.

During development, the major issue was achieving the proper clock speed for our design. Commonly OpenCL designs for Stratix 10 GX FPGAs range from 200-300MHz. Putting the core part of the computation in the clock2x clock domain already improved the performance a lot, where for a single copy of the FullPermutationPipeline we could go up to 550MHz, but once we started filling up more of the FPGA area, performance quickly dropped down to 150-200MHz. It seemed that the compiler was mixing up the hardware for each module. By adding Logic Lock regions to forcibly separate the 10 pipelines' hardware, as shown in Figure 14, we were able to reach our target frequency of 450MHz.

Another issue we encountered was that very rarely a large error would occur during buffer transfer from FPGA memory to main memory. It appeared that multiple pages of data were skipped over, or taken from a different buffer. This turned out to be an issue in the Bittware-provided library for FPGA communication.

In the end we were able to retroactively eliminate all of the instances in which this problem occurred by checking the extra data that was saved along with the main data, namely the counts. This error type that altered whole pages of these buffers was nearly guaranteed to affect the resulting count, and so by checking the count we can check if a subresult was likely to have had an error.

These counts can then also in turn be used to check the validity of the whole computation, by using them to compute $D(n+1)$ instead of $D(n+2)$ as shown in Equation 15. Again, deduplication has to be taken into account for this computation.

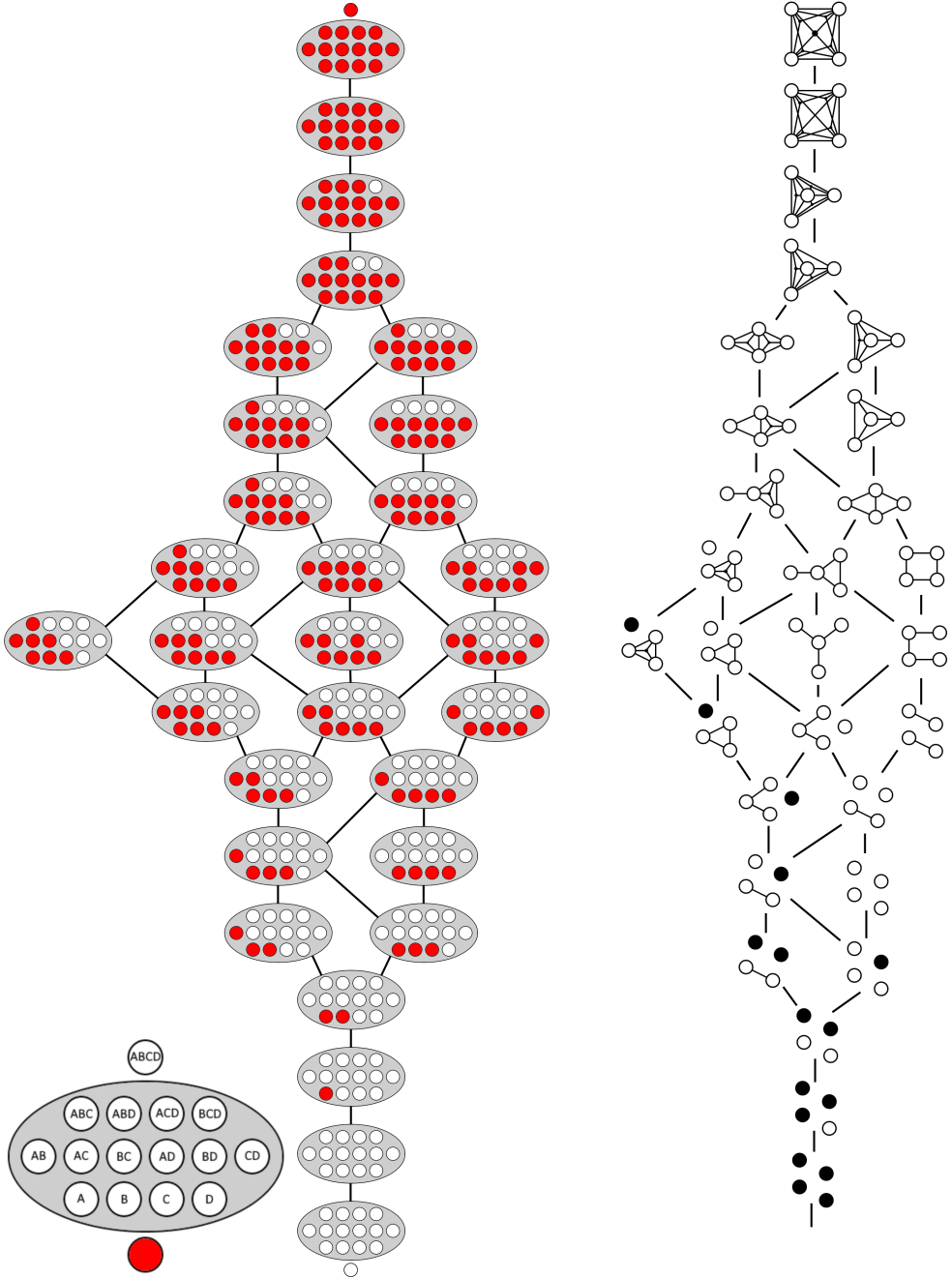


Fig. 12. All equivalence classes in the lattice of 4 Variables. Shown as hypergraphs (right), and a chosen representative (left). Left: for compactness the $\emptyset$ and $\{abcd\}$ nodes are omitted as they always have the same value, except for $\top$ and $\perp$. Right: Singleton hyperedges are represented as empty nodes, hyperedges of arity 3-4 connect 3 nodes with a dot. [8, Figure 6.1]. The implementation uses this graph as a precomputed Directed Acyclic Graph. It is used to efficiently enumerate the β s required in 10 by starting at node α , and systematically following all downwards connections.

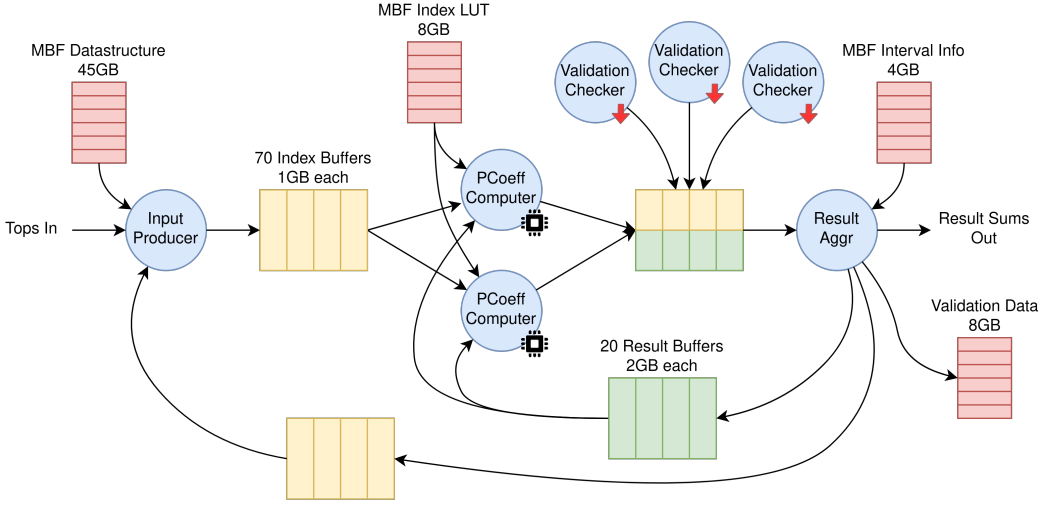


Fig. 13. CPU thread structure. Blue circles represent thread groups, the yellow and green boxes are queues that store buffers to be processed, which travel from thread to thread over the black arrows. Red boxes represent the large static buffers used during processing. They are duplicated for the two NUMA Socket Domains.

$$D(n+1) = \sum_{\alpha \in R_n} E_{\alpha} \sum_{\substack{\beta \in R_n \\ \exists \delta \in \text{Permut}_{\beta}: \alpha \geq \delta}} \frac{E_{\beta}}{n!} \sum_{\substack{\gamma \in \text{Permut}_{\beta} \\ \alpha \geq \gamma}} 1 \quad (15)$$

In order to quickly recompute these term counts, we made use of the Fast Interval Counting algorithm developed by Bartłomiej Pawelski. [15]

7 COMPUTATION ON NOCTUA 2

We implemented this hardware accelerator on the 520N cards operated in Paderborn University's Noctua 2 [1] supercomputer. We were able to fit 300 of these CountConnected Cores on a single field-programmable gate array (FPGA) die. These CountConnected Cores run at 450MHz. This gives us a throughput of about 33 Billion CountConnected operations per second. At this rate, a pair of 520N FPGA cards process about 5.8 α values per second, taking 47'000 FPGA hours to compute $D(9)$ on Noctua 2, or about 3 months real-time on up to 32 FPGAs used concurrently. The full hierarchy is shown in Figure 15.

The computation is split across the system along the lines of Equation 10. α values (also named tops) are divided on the job level. There are 490M tops to be processed for $D(9)$. We split these into 15000 jobs of about 32700 tops each. The β values per top (also named bottoms) are placed in large buffers of 46M bots on average, and sent over PCIe (Peripheral Component Interconnect Express) to the FPGA. The FPGA then computes all 5040 permutations (γ) of each bottom, computes and adds up their P-Coefficients, and stores the result in an output buffer of the same size.

Table 5 summarizes the components and factors that contribute to the overall scale of the project. The upper half of the table designates the huge amount of computational work that was actually performed, as central operation processing a total of around 5.574×10^{18} P-Coefficients (that is 5.574 Exa-P-Coefficients) with the approach described since Section 4.1. In contrast, the lower half of

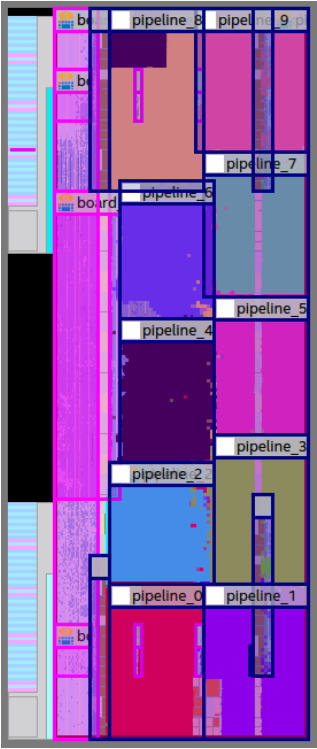


Fig. 14. The FPGA Accelerator Die. The 10 Full Permutation Pipelines we were able to fit on the die are shown in their respective logic lock regions. The magenta shaded boxes on the left are unusable FPGA die space as it was taken up by the Board Support Package, to do things like the DDR memory interface, PCIe interface and OpenCL integration.

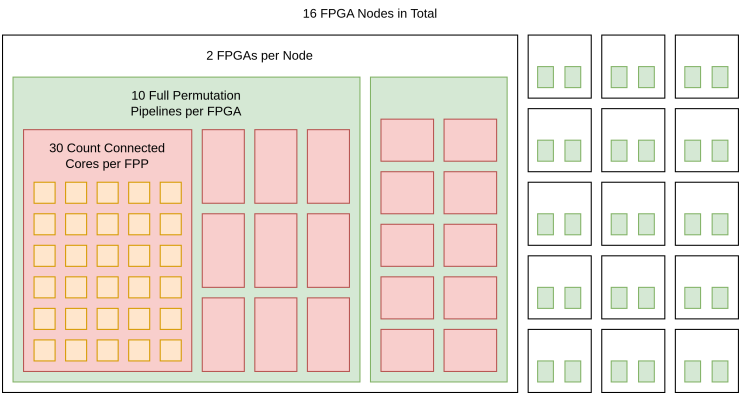


Fig. 15. Hierarchy of the levels of parallelism employed.

the table illustrates the amount of computational work that has been avoided by the mathematical approach (Sections 2, 4) by providing the scale of the factors from Equation 10. Note that the individual factors vary widely within the summation formula and thus can not be multiplied step

	Factor	Value	per FPGA h
Tops α		4.90×10^8	1.04×10^4
Bottoms β per Top	up to 4.90×10^8		
(α, β) Pairs		4.59×10^{16}	
Symmetry Factor	2		
(α, β) Pairs incl symmetry		2.29×10^{16}	4.883×10^{11}
Possible permutations per (α, β)	5040		
Total Permutation #		1.157×10^{20}	2.461×10^{15}
Valid Permutation Fraction	4.96%		
Valid Permutations #		5.574×10^{18}	1.186×10^{14}
P-Coefficients to Compute		5.574×10^{18}	1.186×10^{14}
Connected Components / PCoeff	4.074		
FloodFill Iterations / PCoeff	4.061		
Total FloodFill Iterations		2.264×10^{19}	4.816×10^{14}
$ \perp, \alpha $	up to 2.4×10^{12}		
E_α	up to 5040		
$ \beta, \tau $	up to 2.4×10^{12}		
$E_\beta/n!$	up to 1.0		
$2^{C_{\alpha,\gamma}}$	up to $2^{35} = 3.4 \times 10^{10}$		
D(9)		2.86×10^{42}	

Table 5. Table illustrating the factors that bring the computation of D(9) together. The first section shows the total work size, the second section shows how many iterations it takes to process a single connected component. (Iteration count is not strongly related to the number of Connected Components. Their similarity is merely coincidental, due to it taking multiple cycles per component and the algorithm optimizations employed, See [8] Table 9.3). The third section bridges the gap between the number of terms and D(9) by illustrating the factors involved in the computation. These factors don't factor neatly though, since they are highly correlated. They are merely provided as illustration. Refer to Equation 10

by step as in the upper part of the table, but it is evident that both algorithmic and computational efficiency were of paramount importance for the overall result. Table 6 then dives deeper into the cycle-wise utilization of the components along the FPGA accelerator. It emphasizes how more headroom was created in the data generation and filtering steps, to firmly place the computation bottleneck on the most expensive part of the pipeline: The FloodFill Iterations.

The artifact of this computation is a dataset with an intermediary result for each of the 490M α values. Each of these can be checked separately⁵, and the whole file sums to

$$286386577668298411128469151667598498812366$$

All datasets are available at <https://zenodo.org/records/10579803>.

7.1 Comparison to CPU Compute

In order to assess the FPGA design not only in absolute terms, but also in comparison to a CPU reference, we ran a benchmark for the CPU code on one of the jobs. We used a highly optimized code around manually vectorized components as described in Section 5.3 and Listing 4. Calculating

⁵It takes about 10-2000s to compute a single α result on 128 AMD Epyc CPU cores

	per FPGA h	per FPGA s	max per FPGA clock		Utilization
(α, β) Pairs incl symmetry	4.883×10^{11}	1.356×10^8	10 * 1/7	225 MHz	0.422
Total Permutation #	2.461×10^{15}	6.836×10^{11}	10 * 30 * 24	225 MHz	0.422
P-Coefficients to Compute	1.186×10^{14}	3.392×10^{10}	10 * 30	225 MHz	0.503
FloodFill Iterations	4.816×10^{14}	1.337×10^{11}	10 * 30	450 MHz	0.990

Table 6. Table illustrating the average utilization of components along the FPGA Accelerator. It gives more information on the cycle-wise performance of Table 5. A utilization of ~50% for the data production and filtering components was desired, ensuring that there was no parts except the FloodFill Iteration component would be the bottleneck, irrespective of fluctuations in processing difficulty.

tops is an embarrassingly parallel task, which would make running each top on a single core optimal in terms of minimizing communication between cores. But because running a single top on a single core can take several days, we opted to strike a balance between parallelization within a top, and running several α s in parallel to avoid performance penalties from load-balancing. We divide the 128 cores of a Noctua 2 FPGA node into 16 blocks of 8 cores, corresponding to the core complex size of the AMD Epyc CPUs. Each complex is then responsible for calculating a top, where all cores in the complex work together. This allows for parallelization while still keeping communication overhead negligible. Running one job on 128 AMD Epyc CPU Cores took 150 hours. In this time it had processed 32664 α s. This gives us a computation rate of 0.0604 α s per second. This is 95x slower per node compared to a node of the FPGA implementation⁶. In other words, one Intel Stratix 2800 FPGA outperforms 64 AMD Epyc Milan 7713 cores by a factor of 95x. To calculate D(9) on a CPU cluster in the same time as on 32 FPGAs, 3040 AMD Epyc Milan 7713 CPUs with a total of 194,560 cores would have been required.

8 CORRECTNESS

As much of the code as possible is written generically. This means the same system is used for computing D(3) - D(8). All of these yield the correct results. Of course, the FPGA kernel is written specifically for D(9) computation, so its correctness was verified by comparing its results with the CPU results for a small sample. In effect, both methods verified each other's correctness.

However, we do have several checks to increase our confidence in the result.

- The most direct is the $D(9) \equiv 6 \pmod{210}$ check provided by Pawelski & Szepietowski[16]. Our result passes this check. Sadly, due to the structure of our computation, nearly all terms are divisible by 210, strongly weakening the check. One thing that this check does give us is that no integer overflow has occurred, which was an important concern given we were working with integers of 128 and 192 bits wide.
- As mentioned earlier, our computation was plagued by one issue in particular. We encountered a bug in the Bittware-provided library for communication over PCIe, wherein, occasionally and at a low incidence rate, full 4K pages of FPGA data are not copied properly from FPGA memory to host memory. This results in large blocks of incorrect bottoms for some tops. We encountered this issue in about 2300 tops. We were able to mitigate this issue by including extra data from the FPGA to host memory, namely the 'valid permutation count'. By checking these values, we could determine if a bottom buffer had been corrupted. Additionally, adding all of these counts yields the value for D(8), which shows that the

⁶Earlier texts on this work had mentioned speedups of 500x, but it turned out there was a memory bottleneck giving the CPU an unfair disadvantage. This was fixed for a fairer comparison.

correct number of terms have been added. This issue has at this point been fixed on Noctua 2.

- Finally, there is an estimation formula, which gives us an estimation which is relatively close to our result. The Korshunov estimation formula estimates $D(9) = 1.15 \times 10^{41}$ which is off by about a factor 2.⁷

9 DANGER OF COSMIC RADIATION

The one way our result could still have been wrong was due to a Single Event Upset (SEU), such as a bitflip in the FPGA fabric during processing, or a bitflip during data transfer from FPGA DDR memory to Main Memory.

It is difficult to characterise the odds of these SEU events. Expected number of occurrences for the FPGAs we used are not available to the best of our knowledge. But example values shown on Intel's website pin the error rate at around 5000 SEU events per billion FPGA hours. In that case, given our 47000 FPGA hours, we expect to see 0.235 Poisson distributed errors, giving us a chance of 20% of a hit. Of course, this is just an example and the real odds might have been higher than that.

We did work to harden the accelerator against such bit errors to the extent that this was feasible. Intel Stratix' M20k blocks come equipped with built-in error detection and correction if enabled. We enabled the error detection, and thereby rejected any run that encountered such a bit error. Not a single one happened though during our run, which gives more confidence that the error rate is relatively low. As for the program CRAM it is difficult to tell if it came pre-hardened out of the box. The Intel Stratix chips all provide an automatic round-robin test of the CRAM contents, and while this information is available on a package pin, Bittware's documentation did not mention it, so we don't know if they implement it or not. Finally the contents of registers in the logic are near-impossible to harden, so we didn't try.

In the end, the most powerful confirmation that no bit errors occurred was that our result was identical to Jäkel's[10].

10 CONCLUSION

In this paper we tackled the problem of computing the 9th Dedekind number. We first provided a mathematical framework describing Monotone Boolean Functions and the operations that can be performed on them. Then we went into detail on how the implementation works, starting from the key advantage that made FPGAs a strong choice for the algorithm we wished to use. We build up the FPGA accelerator from this building block up the hierarchy for data flow management and parallelization. Once we had a full FPGA accelerator, we describe the steps that needed to be taken to produce the data at the rate that the accelerator consumes it. Finally, we give the results of actually executing the compute project, and show the observed advantage against what it would have been, had we implemented the computation on CPU.

So what does the future hold for the 10th Dedekind number? Computing that will require a change in methods, due to the sheer increase in computational labor that would need to happen. The fundamental complexity of our algorithm was on the order of $O(R(n-2) \times D(n-2))$. For our implementation this amounted to 4.59×10^{16} terms, for $D(10)$, that would transform to something in the ballpark of 10^{34} terms, far exceeding current computational ability. Also Jäkel's approach would crumble under this demand, the computational complexity is just too great.

⁷This isn't too unusual though, as the results for odd values are off by quite a lot. Estimation for $D(3)$ overestimates by a factor 2, $D(5)$ also overestimates by a factor 2, and $D(7)$ overestimates roughly 10%

Our result was originally uncertain due to the possibility of bit errors due to cosmic radiation. We held off on publication until we had completed the second run. although we did register the value we got in the corresponding github commit from March 9, 2023:

<https://github.com/VonTum/Dedekind/commit/1cf7b019afca655586e8210f97fbb5399d61e842>

On April 4, 2023, a preprint claiming $D(9)$ was published “A computation of the ninth Dedekind number A Preprint” by Christian Jäkel[10], which arrived at the exact same number we had. Since this proved the correctness of our own result, we decided to publish our own preprint as well [9]. All code is available at <https://github.com/VonTum/Dedekind>. All data is available at <https://hirtum.com/dedekind>.

ACKNOWLEDGMENTS

The authors gratefully acknowledge the funding of this project by computing time provided by the Paderborn Center for Parallel Computing (PC2).

REFERENCES

- [1] Carsten Bauer, Tobias Kenter, Michael Lass, Lukas Mazur, Marius Meyer, Holger Nitsche, Heinrich Riebler, Robert Schade, Michael Schwarz, Nils Winnwa, Alex Wiens, Xin Wu, Christian Plessl, and Jens Simon. 2024. Noctua 2 Supercomputer. *Journal of large-scale research facilities (JLSRF)* (2024). <https://doi.org/10.17815/jlsrf-8-187>
- [2] Patrick De Causmaecker and Lennart Van Hirtum. 2024. Solving systems of equations on antichains for the computation of the ninth Dedekind Number. arXiv:2405.20904 [math.CO]
- [3] Patrick De Causmaecker and Stefan De Wannemacker. 2011. Partitioning in the space of antimonotonic functions. arXiv:1103.2877 [math.NT]
- [4] Patrick De Causmaecker and Stefan De Wannemacker. 2014. On the number of antichains of sets in a finite universe. arXiv:1407.4288 [math.CO]
- [5] Patrick De Causmaecker, Stefan De Wannemacker, and Jay Yellen. 2016. Intervals of Antichains and Their Decompositions. arXiv:1602.04675 [math.CO]
- [6] R. Dedekind. 1897. *Über Zerlegungen von Zahlen Durch Ihre Grössten Gemeinsamen Theiler*. Vieweg+Teubner Verlag, Wiesbaden, 1–40. https://doi.org/10.1007/978-3-663-07224-9_1
- [7] J. Deepakumara, H.M. Heys, and R. Venkatesan. 2001. FPGA implementation of MD5 hash algorithm. In *Proc. Canadian Conf. on Electrical and Computer Engineering (CCECE)*, Vol. 2. 919–924 vol.2.
- [8] Lennart Van Hirtum. 2021. A path to compute the 9th Dedekind Number using FPGA Supercomputing. <https://hirtum.com/thesis.pdf>. <https://hirtum.com/thesis.pdf> KU Leuven, Masters Thesis.
- [9] Lennart Van Hirtum, Patrick De Causmaecker, Jens Goemaere, Tobias Kenter, Heinrich Riebler, Michael Lass, and Christian Plessl. 2023. A computation of $D(9)$ using FPGA Supercomputing. arXiv:2304.03039 [cs.DM]
- [10] Christian Jäkel. 2023. A computation of the ninth Dedekind number. *Journal of Computational Algebra* 6-7 (2023), 100006. <https://doi.org/10.1016/j.jaca.2023.100006>
- [11] The On-Line Encyclopedia of Integer Sequences® (OEIS®). 1964. A000372, *Dedekind numbers or Dedekind’s problem*. <https://oeis.org/A000372> Consulted on 16/10/2023.
- [12] The On-Line Encyclopedia of Integer Sequences® (OEIS®). 1964. A003182, *Dedekind numbers:inequivalent monotone Boolean functions of n or fewer variables*. <https://oeis.org/A003182> Consulted on 16/10/2023.
- [13] Jan-Oliver Opdenhövel, Christian Plessl, and Tobias Kenter. 2023. Mutation Tree Reconstruction of Tumor Cells on FPGAs Using a Bit-Level Matrix Representation. In *Proceedings of the 13th International Symposium on Highly Efficient Accelerators and Reconfigurable Technologies* (Kusatsu, Japan) (HEART ’23). Association for Computing Machinery, New York, NY, USA, 27–34. <https://doi.org/10.1145/3597031.3597050>
- [14] Bartłomiej Pawelski. 2021. On the number of inequivalent monotone Boolean functions of 8 variables. <https://doi.org/10.48550/ARXIV.2108.13997>
- [15] Bartłomiej Pawelski. 2023. Counting interval sizes in the poset of monotone Boolean functions. arXiv:2311.11744 [math.CO]
- [16] Bartłomiej Pawelski and Andrzej Szepietowski. 2023. Divisibility properties of Dedekind numbers. arXiv:2302.04615 [math.CO]
- [17] Heinrich Riebler, Tobias Kenter, Christian Plessl, and Christoph Sorge. 2014. Reconstructing AES Key Schedules from Decayed Memory with FPGAs. In *Proc. IEEE Symp. on Field-Programmable Custom Computing Machines (FCCM)*. IEEE Computer Society, Washington, DC, USA, 222–229.

- [18] Heinrich Riebler, Tobias Kenter, Christoph Sorge, and Christian Plessl. 2013. FPGA-accelerated Key Search for Cold-Boot Attacks against AES. In *Proc. Int. Conf. on Field Programmable Technology (ICFPT)*. IEEE Computer Society.
- [19] Heinrich Riebler, Michael Lass, Robert Mittendorf, Thomas Lücke, and Christian Plessl. 2017. Efficient Branch and Bound on FPGAs Using Work Stealing and Instance-Specific Designs. *ACM Trans. Reconfigurable Technol. Syst.* 10, 3, Article 24 (jun 2017), 23 pages. <https://doi.org/10.1145/3053687>
- [20] Tamon Stephen and Timothy Yusun. 2014. Counting inequivalent monotone Boolean functions. *Discrete Applied Mathematics* 167 (2014), 15–24. <https://doi.org/10.1016/j.dam.2013.11.015>
- [21] Robert Tarjan. 1972. Depth-First Search and Linear Graph Algorithms. *SIAM J. Comput.* 1, 2 (1972), 146–160.
- [22] Doug Wiedemann. 1991. A computation of the eighth Dedekind number. <https://link.springer.com/article/10.1007%2F00385808>. <https://link.springer.com/article/10.1007%2F00385808>